1 Robot movement planning

A set of boxes of size $x \times y$ are received as a continuous stream over a belt (the side y is orthogonal to the movement). A disposition of n boxes to form a packaging layer is given (each box i is associated to a destination position $P_x[i], P_y[i]$), so that boxes are not overlapping and typically tile a rectangle with potential gaps around.

A robotic arm is capable of picking a set of boxes from the stream (assume no gaps between boxes when they are picked). Given the gripper size S, the gripper can pick at most boxes of their x-axis with a total length of $S + x$, with at most $x/2$ package overflow from each side. Moreover the gripper can move only one of the two parallel plates to pick and release objects. Therefore when opening, it must be checked that no collisions are created with existing objects. See `youtu.be/oXiIPEDNTF8?si=DSnOmsa2VyUyBnZq` for an example of more advanced features

1. Write a Minizinc program capable of finding a sequence of actions that pick a selected number of boxes and release them into final positions as described by P positions.

2. The problem can be solved by two intermediate modelings. The first one identifies the maximal sets of boxes on the pallet that can be handled with a single pick, depending on their alignments. The second one defines the order in which single manipulations (drops on the pallet) are scheduled for loading the entire pallet, taking into account the gripper movement for releasing the boxes.

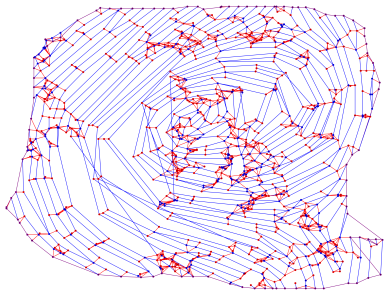
3. Prepare a set of benchmark instances in the following way. For increasing n and decreasing x, y use `www.onpallet.com/` to randomly generate 5 different instances for each pair x, y for suitable n. The size S can be set at most equal to $3x$. In total you should produce 20 instances.

4. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc).

5. Write a 6–10 pages report containing your models (and the reasons for your choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

2 Graph sub isomorphism

In digital forensics the identification of fingerprints can be achieved with graph analysis. Each fingerprint can be abstracted by a graph, where nodes correspond to *minutiae* (ending of ridges and/or bifurcation of ridges). Ridges are the lines that are that are more distant from fingers and form a pattern that can be observed on a surface touched by the finger.



Comparing a pair of fingerprints can be performed on two corresponding graphs G1 and G2. The goal is to find one or many common sub isomorphisms. For each node on G1 there may be a node on G2 with the same characterisation and with same neighborhood. Nodes are classified according to type (ending, bifurcation) and according to relative position of nodes. Two nodes in G1 are in relation according to their connectivity on the graph. In order to account for close-by nodes on parallel ridges, additional nodes (added) are introduced in the graph, so that graph connectivity allows to reach all nodes.

1. Write a Minizinc program capable of finding the set of graph sub isomorphisms, given two graphs (preprocessing for input data of json input files can be applied).
2. The program can compute all sub isomorphisms and return for each solution a possible isomorphism
3. The benchmark set is uploaded on Elly
4. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc).
5. Write a 6–10 pages report containing your models (and the reasons for your choices) and a presentation of the execution results. Prepare the programs in a unique zip file.

3 Word sorting

Natural language hides many rules for the correct order of words in a sentence. The goal is to develop a CSP that provides as solution the correct order for words, depending on the type of each word.

Assume to have a noun and its attributes. You can apply different constraints for English rules and Italian rules.

1. Write a Minizinc program capable of sorting a set of words, according to the input type.
2. The first scenario is the one where a noun is input together with an optional set of article, adjective, adverb, number (absolute, ordinal), unit of measure. You may want to consider sub categories, depending on semantics of words. There is no need to fix declinations, just the order is needed.
3. The second scenario is the one where verbs and their related words are handled. In this case you can assume to have potential auxiliary verb, adverbs, etc.
4. Prepare your model for both languages and define the types and subtypes of words. Define your CSP rules in the most general way, so that to cover as many cases as you can.
5. Write a 6–10 pages report containing your models (and the reasons for your choices) and a presentation of the execution results. Prepare the programs in a unique zip file.

4 Tiling a chessboard

Consider a $n \times n$ chessboard where some cells are excluded (*walls*) by means a (possibly empty) array of pairs [i,j] with $i, j \in \{1, \ldots, n\}$.

You have as many as you need "L" and "Γ" pieces plus unitary "U" pieces. ("L" and "Γ' are made by 4 unitary pieces each—see figure). The goal is to cover the chessboard with the pieces in such a way to partition the chessboard, with no overlapping pieces, avoiding the walls and minimizing the number of "U" to be used.

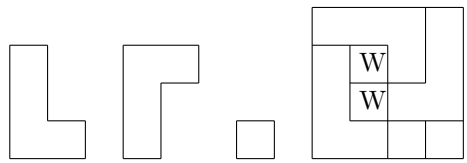


Figure 1: "L", "Γ", and "U" pieces and a solution of an instace with $n = 4$ and two walls (denoted by W) in (2,2) and (3,2).

1. Write a Minizinc program capable of finding a solution for an instance of this problem (if any), by *minimizing* the number of unitary pieces used.

2. Prepare a set of benchmark instances in the following way. For $n = 5, 10, 15, 20, 25$ randomly generate 5 different instances with n walls and 5 different instances with $2n$ walls (this can be done using any programming language). You should have 11 instances for any n (including those without walls).

3. Run your Minizinc encoding for all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc) Report the best value for the solution found within the timeout.

4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

5 Sorting as a planning problem

Let us consider a n element vector of numbers $\vec{v} = [v_1, \ldots, v_n]$, where $v_i \in \mathbb{N}$. For $i, j \in \{1, \ldots, n\}$ a $\texttt{swap}(i, j)$ move has the effect of swapping the values $v[i]$ and $v[j]$. The problem is that of finding the plan of minimum length that sorts the vector.

1. Write a Minizinc program that solves the problem.
2. Prepare your benchmark suite as follows. For $n = 5, 10, 15, 20, 25, 30$. For each n compute 10 different permutations of pairs $[1, \ldots, n]$. You have now 60 instance benchmarks.
3. Run your Minizinc encoding on all the instances. Use a timeout of 5 minutes for each test ("configuration" option in Minizinc)
4. Choose one configuration that runs in a couple of minutes and try alternative search primitives on it. Use the most promising one for more difficult instances.
5. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Try to infer some complexity considerations. Prepare the programs and the benchmark instances used in a unique zip file.

6 Evolution

Let us consider two n element vectors $\vec{s} = s_1 \cdots s_n$, and $\vec{t} = t_1 \cdots t_n$, with $s_i, t_i \in \{A, C, T, G\}$. It is conjectured that t is obtained from s by using repeated applications of the action $\texttt{rev}(i, j)$, where $1 \leq i < j \leq n$ that cuts a piece of string, reverses it and insert it back in place. Precisely, if $\vec{a} = a_1 \cdots a_{i-1} a_i \cdots a_j a_{j+1} \cdots a_n$ then the application of $\texttt{rev}(i, j)$ on it returns

$a_1 \cdots a_{i-1} a_j a_{j-1} \cdots a_i a_{j+1} \cdots a_n$.

Given s, t and k the problem is the one of finding a plan of k moves that modifies s to t.

1. Write a Minizinc program that solves the problem.

2. Prepare your benchmark suite as follows. For $n = 10, 15, 20, 25, 30$ generate randomly 5 strings of length n. Then, apply randomly $k = 3, 6, 9, 12$ $\texttt{rev}(i, j)$ moves to each of them (with $j - i$ randomly chosen in $2..n/2$). You have now 100 instance benchmarks.

3. Run your Minizinc encoding on every instance using $k - 2, k - 1, k$ plan lengths (300 runs). Use a timeout of 5 minutes for each test ("configuration" option in Minizinc)

4. Choose one configuration that runs in a couple of minutes and try alternative search primitives on it. Use the most promising one for more difficult instances and report what happens.

5. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

7 Industry shifts

There are n workers, each of them with a rank 1–4 (given as input). There are k machines, each of them associated to one of the types 1, 2, 3 (given as input).

Rank 1 workers can work only at machines of type 1. Rank 2 workers can work only at machines of type 1 or 2. Rank 3 workers can work only at machines of type 1 or 3. Rank 4 workers can work at every machine.

The goal is to produce a weekly schedule for workers over the k available machines, according to the admissible mappings described above. Possible work shifts are: 6–14, 14–22, 22–6, 8–16, 16–24, 24–08. Each worker has a constraint over consecutive shifts: there must be at least 24 hours between the two shifts *starts*. Scheduling is cyclic: Sunday's shifts must be compatible with Monday's ones. Each worker has two days off per week; if he/she works on Saturday/Sunday night, there are 3 days off per week. If he/she works on Friday night, he/she can't work on Sunday. Each worker covers from 32 to 36 hours per week. Shifts length for a worker is 8 hours, but at most once a week it can be reduced to either 4 or 6 hours.

Machines of type 3 can't be used for shifts starting at 6 (OK for shifts starting at 8). Machines of type 2 can't be used for shifts starting at 24 (OK for shifts starting at 22).

Find a solution that maps workers to machines so that the machine usage is maximized.

1. Write a Minizinc program capable of finding an optimal solution to an instance of this problem (if any).

2. Prepare a set of benchmark instances in the following way. Generate a set of 50 benchmarks of increasing size. Choose some input numbers n, k that ensure satisfiability (save for 5 cases). Balance the number of worker ranks and machine types.

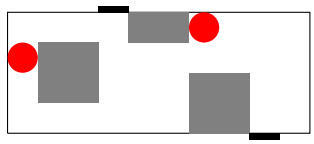
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc) Report the best value for the solution found within the timeout.

4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

8 The great escape

Let us define a rectangle of size $m \times n$ with a set of excluded smaller squares $s \times s$. The input provides an array of square sizes and positions. Squares are non walkable areas.

Along the rectangle sides there are k different doors, facing one border cell. Finally, there are k players assigned to different available cells in the rectangle. At a certain time step t, each player can move to a neighbor cell. The player can't move to a cell that is occupied at time $t + 1$ by another player (or by an excluded cell). Therefore, at time step t from 0 to k players can move.



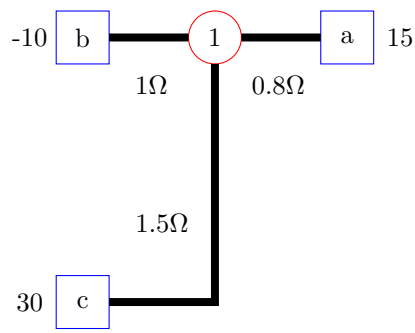
In the figure, non square shapes gray areas are made of a combination of excluded squares. The doors are highlighted by a thick black line and players are the red symbols.

The problem is, given a time t, to find an escape plan that allows to every player to exit from the doors. As an additional rule, no door can be used by multiple players, even at different times. There is no pre-determined association between doors and players.

1. Write a Minizinc program capable of finding a solution of the above problem (if any)
2. Prepare a set of benchmark instances in the following way. Choose the size (m, n) (start from small values as in the example above), for instance $(5, 3), (10, 4), (15, 5)$. For each size generate 5 instances of the "room" For each of them generate 4 instances with 1,2,3, and four 4 people/doors (random positions). You should have 240 benchmark instances.
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc).
4. For instances with reasonable running time, compute the minimum time t needed to solve the problem
5. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

9 Energy saving

Let us model an energy distribution grid by a graph. There are two types of nodes: type 1 node is an energy producer/consumer (e.g. solar home, which sometimes produces more than it consumes). A simplified model accounts for electrical current: a negative value (e.g., -10) defines a request of 10 Amperes and a positive value (e.g., 4) defines a possible energy production of 4 Amperes. Type 2 node is a current switch that selectively connects exactly 2 out of 3 attached lines. As an example in the figure, it can connect left and right lines, left and bottom lines, right and bottom lines.



In the figure, node b requests 10 Amperes of current, that can be sent from a and c. If the switch connects node c, there is a power dissipation of

$$10^2 \cdot 1.5 + 10^2 \cdot 1 = 250W$$

(recall that dissipation depends on I^2R, where I is the current in Amperes and R is the resistance of the lines in Ω). If the switch connects node a instead, there is a power dissipation of

$$10^2 \cdot 0.8 + 10^2 \cdot 1 = 180W$$

The third switch combination (the second one) doen't produce any effect for node b.

Given a grid network, the goal is to determine whether is possible to satisfy all requests and find switch configurations that minimize dissipation.

1. Model this problem with Minizinc.
2. Prepare a set of benchmark (100) instances of increasing number of nodes. Work a bit in order they are meaningful.
3. Run your Minizincencoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc)
4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

10 Autonomous Car Sharing

In 2026 autonomous cars will offer an optimized car sharing service in Parma. There are k cars with 4 seats available.

By the night before, all the requests for the next day are collected. Next morning, the app will notify users the schedule.

Requests are structured with the following form: I need to go at x by time y and I will stop there for z hours before going back home. Let us assume there are t users.

Suppose that all trips start from Piazza Garibaldi in Parma that will become the main hub.

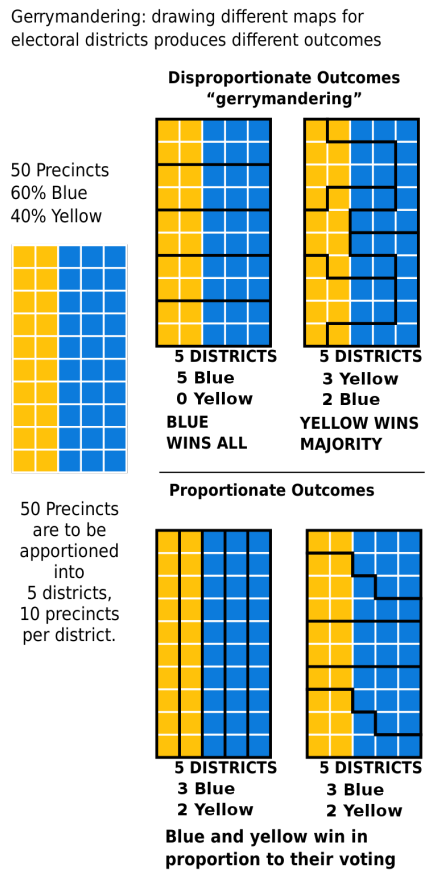
The problem goal is to find a mapping between cars and users such that: (1) all users are brought to destination within requested times (the arrival is at most half an hour before requested time y and departure is at most half an hour later that time $y + z$) and (2) minimizes the sum of travelled distance by cars (if each car is used by a single user the solution is not optimal).

For a real use-case, build a graph that connects real landmarks in the town and annotate distances on edges (from google maps). Assume that cars offer an average speed of 20km/h.

1. Write a Minizinc program capable of solving the above problem.
2. Prepare a set of 100 benchmark instances with increasing k, t and different daily requests.
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc).
4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

11 Gerrymandering

It is voting times. Let us receive as input a geographic rectangular region M $m \times n$, where each cell represents a voter. There are two parties (coded with 0 and 1). The matrix M is filled by 0 and 1, according to the vote forecast for each voter, thanks to social media analysis. There is a number of k districts (a set of contiguous voters in the matrix M). The system elects the representative for each district, according to the majority of votes in the district. Both parties has the (opposite) goal to design the k districts so that the number of their representatives is maximied.



1. Write a Minizinc program capable of solving the above problem, finding the best district allocation for both the parties (two solutions, one for each party).
2. Prepare a set of 100 benchmark instances with increasing m, n, k and different input matrices.
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc).
4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

12 Neural

A number of k neurons is given as input. Neurons output a value $n_1, \ldots, n_k$. Neurons are connected according to a binary relation R. The $k \times k$ input matrix R contains $R[i,j] = 1/-1$ if there is a direct arc from n_i to n_j with a positive/negative contribution. Each neuron n_i has a weight p_i, given as input, and it can be set up as off, on and/or enhanced. If it is off, its output is equal to 0. If it is on, its output is equal to p_i multiplied by the sum of incoming values, i.e. the output by any neuron j multiplied by $R[j,i]$. If it is enhanced, the output is equal to p_i multiplied by the sum of squared incoming values. At most $\lceil k/10 \rceil$ neurons can be enhanced.

Neurons with no incoming edges (sensors) are inputs (assume they are at the beginning $n_1, n_2, \ldots, n_h$). We have two output neurons (let us assume n_{k-1}, n_k).

Given an assignment of values (negative/zero/positive) to input neurons $n_1, n_2, \ldots, n_h$, the problem is to determine the configuration of off/on/enhanced neurons that maximizes $|n_{k-1} - n_k|$.

1. Write a Minizinc program capable of finding the optimal solution.
2. Prepare a set of benchmark instances in the following way. For each $k = 20, 25, 30, 35, 40$ and $h \approx k/3$ randomly generate 20 different instances (p_i and R). (100 instances in total)
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc) Report the best value for the solution found within the timeout.
4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

13 2D Packing

A number of boxes with size $x \times y$ must be loaded on a pallet of size $k \times k$. Boxes can be placed so that no boxes overlap and no boxes overflow the pallet. Boxes can be rotated by 90 degrees and they have one side that is parallel to one pallet's side. Assume to work with integer coordinates.

The problem goal is to maximize the number of boxes that can be loaded on the pallet and to produce an arrangement in the 2D space.

1. Write a Minizinc program capable of finding the optimal solution.
2. Prepare a set of benchmark instances in the following way: $x = 5, 10, 15, 20$ and $y = x + 5$ box size and $k = 20 - 1, 30 - 1, \ldots, 100 - 1$. (36 instances in total)
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc) Report the best value for the solution found within the timeout.
4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

14 Campus timetabling

Assume to have a set of classrooms $1..k$ with capacity $c_1, \ldots, c_k$. There is a set of teachers $1..t$ and a set of classes $1..c$ to be taught. The relation $T(i, j)$ with $i \in 1..t, j \in 1..c$ specifies that the teacher i teaches the class j. Each class j has an associated number of students s_j and a number of hours per week (either 4 or 6). Classrooms can be used for 4 slots during each day (20 slots in total). Each slot occupies 2 hours. Each year of a degree program consists of a number of classes in the set $1..c$ (roughly we have $c/9$ degrees). Trivial constraints request teacher to teach at most one class at the time, students of the same year can't attend more than one class at the time and a classroom can't host more than one class at the time.

The problem is to compute a weekly schedule that accounts for the following soft constraints: (1) students weekly schedule should minimize the number of empty slots between full slots; (2) students should have a day free from lessons (3) teachers can express preferences for slots as a binary matrix. Value 0 means no preference, 1 is a preference to teach during that slot, -1 is a preference not to teach during that slot. Preferences have to be maximized

1. Write a Minizinc program capable of finding the optimal solution.
2. Prepare a set of benchmark instances in the following way: $k = 2, 5, 10$, $t = 5, 10, 15$, $c = 9, 18$. (36 instances in total). Generate some teacher preferences with random distribution.
3. Run your Minizinc encoding on all the instances, possibly exploring different search strategies, with a timeout of 5 minutes for each test ("configuration" option in Minizinc) Report the best value for the solution found within the timeout.
4. Write a 6–10 pages report containing your models (and the reasons for some choices) and a presentation of the execution results. Prepare the programs and the benchmark instances used in a unique zip file.

15 Randori

A Judo gym wants to plan its Judo training sessions. The club has a tatami that can simultaneously host either k competitive "adult" athletes, or $\frac{3}{2}k$ competitive "youth" athletes, or $2k$ non-competitive (children) athletes. The groups are distinct, with no mixing.

The instructor can start training at 16.00, except on Saturday, which may be used in the morning only by competitive adults. Thus, consider three gross 2-hour slots from Monday to Friday, and two slots on Saturday morning.

The athletes are $a_1, \ldots, a_m$, $r_1, \ldots, r_n$, $b_1, \ldots, b_p$; each athlete has a weight category w and a belt c. Assume that m, n, p are sometimes larger than k (especially p).

Organize the weekly training schedule in such a way as to satisfy the following constraints:

- Within each group, each week everyone must be able to do *randori* (practice bout) with all those in their weight category or in the category immediately below.
- Within each group, each week everyone must be able to do randori on two days with all those in their weight category and belt, or in the belt immediately below.
- Each competitive adult must be able to train at least 4 times, each competitive youth at least 3 times, and each non-competitive athlete at least 2 times.
- Non-competitive athletes must have at least one rest day between two training sessions.
- If there is not enough room for everyone on Saturday morning, those who do not have a slot on Saturday must train on Friday.
- Only adults can train in the third slot. The instructor does not want empty intermediate slots in the gym (i.e., if sessions start from the second slot, or if the third slot is not used). Each day must always have at least two slots (preferably three, except on Saturday).

1. Write an Minizinc program that models and solves the problem.
2. Prepare benchmark datasets as follows. Generate 100 combinations of instances of groups, weights, and belts. Choose k so that it is reasonable with respect to the parameters.
3. Run the Minizinc program on all 100 test cases. Use a timeout of 5 minutes (e.g., with options in Minizinc IDE).
4. Choose a configuration that requires a couple of minutes to find the solution. Try different solver configurations. Select the best configuration for solving the most difficult instances and document the improvement.
5. Write a 6–10 page report describing the models, the implementation choices, the results, and the execution times. Prepare programs, datasets, and report in a zip file.

16 Trucks

Let $G = (V, E)$ be an undirected graph. There are h identical objects placed on a set $S \subseteq V$ of nodes. Each node in S may contain more than one object, but contains at least one. A node in V may contain 0, 1, or more objects. A truck driver initially stands on node $u \in V$.

The goal is to move all the objects to a set of nodes $T \subseteq V$. Each node in T may contain more than one object, but must contain at least one.

The driver may load an object while passing through a node (entering and then leaving the node). The truck can hold at most two objects at the same time. However, if it already contains two objects, it cannot traverse a node that contains another object. There are 3 actions: `load`, `unload`, and `cross`(x, y) with $x, y \in E$.

1. Write a Minizinc program that models and solves the problem by searching for a plan of length k that reaches the goal.

2. Prepare the benchmark datasets as follows. Create (randomly or inspired by real graphs) graphs of size $|V| = 10, 14, 18, 22, 26$. Choose $h \simeq |V|/5$ and choose T at random ($|T| \simeq 3..6$). Repeat these choices 4 times. Start from a consistent solution and simulate k random object movements ($k = 5, 8, 11, 14, 17$) and create the set S. Choose the driver's starting point at random. In this way you obtain $5 \cdot 4 \cdot 5 = 100$ instances to use for testing.

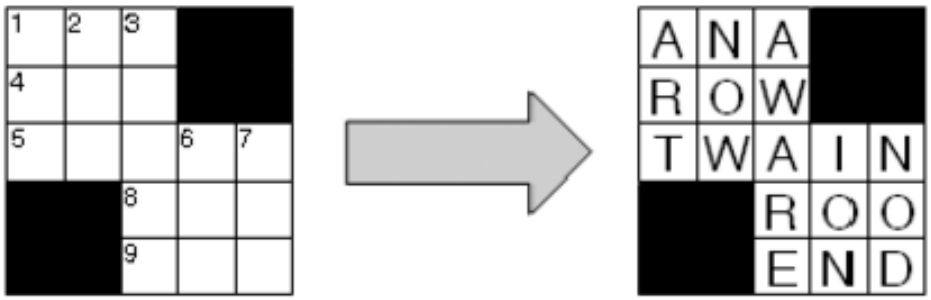
3. Run the Minizinc program on all 100 test cases, trying to increment the values of k (starting from $k = 1$). Use a timeout of 5 minutes for each test, until the solution is found or until the value of k used in the construction of the instance is reached.

4. Choose a configuration that requires a couple of minutes to find the solution. Try different solver configurations. Select the best configuration for solving the most difficult instances and document the improvement.

5. Write a 6–10 page report describing the models, the implementation choices made, the results, and the obtained runtimes. Prepare programs, datasets, and report in a zip file.

17 Crosswords

The goal is to find the solution of a crossword puzzle starting from the set of words to be inserted. For example, the set of words $\{ANA, ROW, TWAIN, ROO, END, ART, NOW, AWARE, ION, NOD, WHITE, BLUE, GREEN, RED, BLACK\}$



The dimensions of the rectangle m and n (in this case 5×5) are given as input. In this example, only 10 out of 15 words were used and 8 cells are marked in black.

The task is to select words from the set `WORDS` in such a way as to minimize the function

$$f(sol) = 2 * \text{number of black cells} - \text{number of used words}$$

1. Write a Minizinc program that models and solves the problem by searching for a plan of length k that reaches the goal.
2. Prepare benchmark test cases with 20 crossword puzzles (search online for solutions) of increasing size.
3. Run the Minizinc program on all instances (create 5 variants for each instance by changing the size of the rectangle and adding some extra words). Use a timeout of 5 minutes for each test.
4. Choose a configuration that requires a couple of minutes to find the solution. Try different solver configurations. Select the best configuration for solving the most difficult instances and document the improvement.
5. Write a 6–10 page report describing the models, the implementation choices made, the results, and the obtained runtimes. Prepare programs, datasets, and report in a zip file.

18 Transit Line Frequency Optimization

A city transit network has l lines. Each line i has a maximum allowed frequency $f_i^{\max}$ and cost per trip c_i. Passenger demand across segments is given as $d_{u,v}$ for all $u, v \in P$, where P is the set of stops and (u, v) is a pair of consecutive stops on a line. Optimize the frequency for each line i so that demands are met and costs are minimized.

Use the OpenStreetMap (OSM) transport networks a city (preprocessed into a graph). Demand matrices can be retrieved from open transit datasets (e.g., GTFS feeds).

1. Implement a MiniZinc model using linear constraints for capacity and cost.
2. Compute optimal frequencies for each city network.
3. Vary objective weights (minimize cost vs. meet demand).
4. Test with 20 synthetic instances generated from GTFS data.
5. Write a 6–10 page report describing the models, the implementation choices made, the results, and the obtained runtimes. Prepare programs, datasets, and report in a zip file.

19 Sudoku Instance Generation With Uniqueness Guarantee

The goal of this project is to automatically generate Sudoku instances of the standard size 9×9 that:

- have at least one valid solution;
- have exactly one admissible solution;
- contain as few clues (pre-filled cells) as possible while preserving uniqueness.

.

The final instances must be generated and then validated through a MiniZinc model.

For experiments and for comparing the quality of the generated instances, use a list of complete Sudoku solutions taken from public datasets: Kaggle Sudoku Dataset (`https://www.kaggle.com/datasets/rohanrao/sudoku`).

1. Implement a MiniZinc model using linear constraints for capacity and cost.
2. Measure time vs required clues.
3. Evaluate different strategies for testing the uniqueness of the solution.
4. Write a 6–10 page report describing the models, the implementation choices made, the results, and the obtained runtimes. Prepare programs, datasets, and report in a zip file.

20 Implement your ASP project

If you already presented a project for the Declarative Programming class, you can rewrite your solution in Minizinc and present a comparison of running times on the same benchmark set. Test whether any strategy for CSP encoding can be useful to improve efficiency.